

Generative Models: Autoencoders & VAEs

1 Autoencoders and the Bottleneck

An autoencoder is a neural network that learns to copy its input to its output, but with a twist: in the middle, it must squeeze the information through a narrow layer.

It has two halves:

- **Encoder** f_ϕ : takes input x and compresses it into a small code $z = f_\phi(x)$.
- **Decoder** g_θ : takes the code z and tries to rebuild the original input, giving $\hat{x} = g_\theta(z)$.

Training simply tries to make $\hat{x}$ look like x . For images or other continuous data, we usually minimize the squared error $\|x - \hat{x}\|^2$. For pixel values treated as probabilities (0 to 1), binary cross-entropy is used instead.

Why a bottleneck? The code z has far fewer dimensions than x . To still reconstruct x well, the network is forced to throw away unimportant details and keep only the most useful information. This is essentially a learned, nonlinear version of PCA – a way of compressing data and extracting useful features.

1.1 The Problem with Plain Autoencoders

A normal autoencoder only learns to map specific inputs to specific codes. It gives no guarantee about what happens *between* those codes. So:

- Two nearby points in the code space might decode to completely different, meaningless outputs.
- You cannot just pick a random z and expect a realistic output.

In short, the latent space has no nice structure – so we can't use it to *generate* new data. This is exactly the gap that Variational Autoencoders fill.

2 Variational Autoencoders (VAEs)

A VAE turns the autoencoder into a proper generative model by adding probability into the picture.

Instead of the encoder outputting one fixed code z , it now outputs the **parameters of a distribution** over possible codes – usually a Gaussian with mean $\mu_\phi(x)$ and variance $\sigma_\phi^2(x)$. We write this distribution as $q_\phi(z | x)$.

The decoder still maps a code z back to data, but now it defines a distribution over outputs, $p_\theta(x | z)$. We also pick a simple, fixed prior for the codes:

$$p(z) = \mathcal{N}(0, I)$$

The generative story: to create new data, first sample a code $z \sim p(z)$, then sample an output $x \sim p_\theta(x | z)$.

The training goal is to make the model assign high probability to the real data, i.e. maximize

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz.$$

This integral is impossible to compute directly (it would mean checking every possible z). The VAE's trick is to bring in the encoder $q_\phi(z | x)$ as a stand-in for the true (but unknown) posterior, and instead optimize a quantity we *can* compute: the **ELBO**.

3 Deriving the ELBO

We start from the log-probability of the data, $\log p_\theta(x)$, and rewrite it by multiplying and dividing by $q_\phi(z | x)$ inside the integral:

$$\log p_\theta(x) = \log \int p_\theta(x, z) dz = \log \int q_\phi(z | x) \frac{p_\theta(x, z)}{q_\phi(z | x)} dz = \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x, z)}{q_\phi(z | x)} \right]$$

Step 1: Apply Jensen's inequality. The log function is concave, so $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$. Applying this gives:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z | x)} \right] \equiv \text{ELBO}$$

This right-hand side is the **Evidence Lower Bound (ELBO)** – a quantity that is always less than or equal to $\log p_\theta(x)$, but unlike $\log p_\theta(x)$, we can actually compute and optimize it.

Step 2: Split it into two interpretable pieces. Using $p_\theta(x, z) = p_\theta(x | z) p(z)$:

$$\begin{aligned} \text{ELBO} &= \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{q_\phi(z | x)}{p(z)} \right] \\ &= \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{reconstruction term}} - \underbrace{D_{KL}(q_\phi(z | x) \| p(z))}_{\text{regularization term}} \end{aligned}$$

What each term means:

- **Reconstruction term** – how well the decoder rebuilds x from a sampled code z . Larger is better.
- **KL term** – measures how far the encoder's distribution $q_\phi(z | x)$ drifts from the simple prior $p(z)$. Keeping this small pulls all codes toward a shared, well-organized space, so the latent space stays smooth and usable for generation.

Why is this a valid lower bound? It can be shown that:

$$\log p_\theta(x) - \text{ELBO} = D_{KL}(q_\phi(z | x) \| p_\theta(z | x))$$

The right-hand side is a KL divergence between the encoder's distribution and the true (intractable) posterior. Since KL divergence is always ≥ 0 , the ELBO can never exceed $\log p_\theta(x)$. Maximizing the ELBO therefore does two things at once: it pushes $\log p_\theta(x)$ up, and it pulls $q_\phi(z | x)$ closer to the true posterior $p_\theta(z | x)$.

In practice, we minimize the negative ELBO as our loss function:

$$\mathcal{L}(\theta, \phi) = -\text{ELBO} = \underbrace{-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)]}_{\text{reconstruction loss}} + \underbrace{D_{KL}(q_\phi(z | x) \| p(z))}_{\text{KL loss}}$$

computed for each data point and averaged over a batch.

4 The Reparameterization Trick

There's a practical problem: to train with gradient descent, we need to backpropagate through $\mathbb{E}_{q_\phi(z|x)}[\cdot]$, but this expectation involves *sampling* z – and the parameters of that sampling distribution (μ_ϕ, σ_ϕ) depend on ϕ . Sampling itself is a random, non-differentiable operation, so gradients cannot pass through it.

4.1 The Fix

Instead of sampling z directly from $\mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$, we move all the randomness into a separate, fixed source of noise:

$$\epsilon \sim \mathcal{N}(0, I), \quad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$$

Why this works: z is now just a simple, deterministic formula involving μ_ϕ , σ_ϕ , and the random number ϵ . Since ϵ doesn't depend on ϕ , all the randomness is "outside" the network. Gradients can flow normally through μ_ϕ and σ_ϕ during backpropagation, just like any other layer.

Without this trick, we would need much noisier and less efficient gradient estimation methods (such as REINFORCE) to train the model. The reparameterization trick is what makes VAEs trainable end-to-end with standard backpropagation.

5 Summary

- Autoencoders compress data through a bottleneck, but their latent space has no guaranteed structure.
- VAEs fix this by making the encoder output a distribution $q_\phi(z | x)$ and pushing it toward a known prior $p(z)$.
- The ELBO is a tractable lower bound on $\log p_\theta(x)$, made of a reconstruction term and a KL regularization term.
- The reparameterization trick rewrites sampling as a differentiable function plus fixed noise, allowing gradients to flow through the encoder.